

Deploying Solaris on Abacus.AI SuperComputer

Guide for migrating to **always-on, production-grade hosting** on Abacus.AI SuperComputer.

Why SuperComputer?

Current Setup (ChatLLM VM)

-  Great for development & testing
-  Ephemeral (stops when conversation inactive)
-  Preview URL changes with each session
-  Not suitable for real users

SuperComputer Upgrade (\$10/mo)

-  **Always-on** 24/7 hosting
 -  **Persistent** storage & databases
 -  **Custom domain** support (free)
 -  **Production-ready** infrastructure
 -  **SSH access** for full control
 -  **GitHub integration** for CI/CD
-

Prerequisites

1. **Abacus.AI account** with SuperComputer access
 - Sign up at supercomputer.abacus.ai (<https://supercomputer.abacus.ai>)
 - Enable SuperComputer in your account settings (\$10/mo)
 2. **Domain (optional but recommended)**
 - For custom branding (e.g., `solaris.health`)
 - Can use Abacus subdomain instead: `solaris.abacusai.app`
 3. **Code ready**
 - You already have it downloaded! 
-

Deployment Methods

Method 1: Docker Compose (Recommended)

The easiest path — uses the containers we already created.

Step 1: Access SuperComputer

```
# Visit supercomputer.abacus.ai
# Create new project: "Solaris Health"
# You'll get SSH access credentials
```

Step 2: Upload code

```
# Option A: Upload via UI
# - Drag the entire luca-passport folder
# - Or upload the solaris-build-*.tar.gz archive

# Option B: Clone from GitHub
ssh your-supercomputer-url
git clone https://github.com/YOUR_USERNAME/solaris-health.git
cd solaris-health
```

Step 3: Deploy

```
# Set production secrets
export DB_PASSWORD="your_secure_password_here"
export JWT_SECRET="your_secure_jwt_secret_here"

# Deploy with Docker Compose
docker-compose --env-file .env.production up -d

# Seed database
docker-compose run --rm seed

# Check status
docker-compose ps
docker-compose logs -f
```

Step 4: Configure domain

1. In SuperComputer dashboard: **Deploy** → **Custom Domain**
2. Add your domain: `solaris.health`
3. Update DNS at your registrar:
 - **Nameservers** (recommended): Point to Abacus.AI nameservers
 - **CNAME**: Add record → `solaris.health` → `your-app.abacusai.app`
4. Wait for verification (5 mins - 48 hours)
5. Deploy to verified domain 

Step 5: SSL (automatic)

Abacus.AI handles SSL certificates automatically. Your app will be live at:

- `https://solaris.health` (custom domain)
- `https://solaris.abacusai.app` (Abacus subdomain)

Method 2: Native Deployment (Advanced)

For maximum control and optimization.

Step 1: Manual setup

```

# SSH into SuperComputer
ssh your-supercomputer-url

# Install dependencies
sudo apt update
sudo apt install -y postgresql nginx

# Create database
sudo -u postgres psql -c "CREATE USER luca_user WITH PASSWORD 'YOUR_PASSWORD';"
sudo -u postgres psql -c "CREATE DATABASE luca_passport OWNER luca_user;"

# Clone and setup
git clone https://github.com/YOUR_USERNAME/solaris-health.git
cd solaris-health

# Backend
cd backend
npm install --production
psql postgresql://luca_user:PASSWORD@localhost/luca_passport -f schema.sql
psql postgresql://luca_user:PASSWORD@localhost/luca_passport -f schema_solaris.sql
node seed_solaris.js

# Create systemd service
sudo tee /etc/systemd/system/solaris-backend.service << 'EOF'
[Unit]
Description=Solaris Backend
After=network.target postgresql.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/solaris-health/backend
Environment="DATABASE_URL=postgresql://luca_user:PASSWORD@localhost/luca_passport"
Environment="JWT_SECRET=YOUR_JWT_SECRET"
Environment="PORT=5000"
Environment="NODE_ENV=production"
ExecStart=/usr/bin/node src/server.js
Restart=always

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl enable solaris-backend
sudo systemctl start solaris-backend

# Frontend
cd ../
npm install
npm run build

# Serve with Nginx
sudo tee /etc/nginx/sites-available/solaris << 'EOF'
server {
    listen 80;
    server_name solaris.health www.solaris.health;

    # Frontend
    location / {
        root /home/ubuntu/solaris-health/dist;
        try_files $uri $uri/ /index.html;
    }
}

```

```
# Backend API
location /api {
    proxy_pass http://localhost:5000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
}
EOF

sudo ln -s /etc/nginx/sites-available/solaris /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

# Get SSL certificate
sudo certbot --nginx -d solaris.health -d www.solaris.health
```

Production Security Checklist

Before going live with real users:

Backend

- [] Change `JWT_SECRET` to cryptographically secure value (32+ chars)
- [] Change database password
- [] Set `NODE_ENV=production`
- [] Enable rate limiting (add `express-rate-limit`)
- [] Configure CORS for your domain only
- [] Set up database backups (daily recommended)
- [] Add error logging (Sentry, LogRocket, etc.)

Frontend

- [] Update `VITE_API_URL` to production URL
- [] Remove demo user auto-fill buttons
- [] Enable analytics (PostHog, Plausible, etc.)
- [] Test all flows end-to-end
- [] Run Lighthouse audit (aim for 90+ scores)

Database

- [] Change all demo user passwords or remove demo users
- [] Set up automated backups
- [] Enable point-in-time recovery
- [] Restrict database access to localhost only
- [] Monitor query performance

Infrastructure

- [] Set up uptime monitoring (UptimeRobot, Better Uptime)
- [] Configure alerts for downtime

- [] Document rollback procedures
- [] Test disaster recovery
- [] Set up staging environment

Monitoring & Maintenance

Health checks

```
# Backend
curl https://solaris.health/api/health
# Should return 200 OK

# Database
psql $DATABASE_URL -c "SELECT COUNT(*) FROM users;"

# Docker (if using containers)
docker-compose ps
docker stats
```

Logs

```
# Docker
docker-compose logs -f backend
docker-compose logs -f frontend

# Systemd
sudo journalctl -u solaris-backend -f
sudo journalctl -u nginx -f
```

Updates

```
# Pull latest code
git pull origin main

# Docker method
docker-compose down
docker-compose build
docker-compose up -d

# Native method
cd backend && npm install --production
sudo systemctl restart solaris-backend
cd .. && npm run build
```

Cost Breakdown

Abacus.AI SuperComputer

- **\$10/month** base subscription
- Includes:

- Always-on compute
- PostgreSQL hosting
- Custom domain (unlimited)
- SSL certificates
- 5GB storage
- GitHub integration

Optional add-ons

- **Domain name:** ~\$12/year (one-time via registrar)
- **Monitoring:** Free tier (UptimeRobot, Plausible)
- **Backups:** Included in SuperComputer
- **Email service:** \$0-20/mo (SendGrid, Postmark for transactional emails)

Total: ~\$10-15/month for production-ready hosting 🎉

Troubleshooting

“Database connection failed”

```
# Check Postgres is running
sudo systemctl status postgresql

# Test connection
psql $DATABASE_URL -c "SELECT 1;"

# Check credentials in .env
cat backend/.env
```

“Port already in use”

```
# Find process on port
sudo lsof -i :5000
sudo lsof -i :3000

# Kill if needed
sudo kill -9 <PID>
```

“Domain not verified”

- Wait 24-48 hours for DNS propagation
- Check nameservers: `dig NS solaris.health`
- Check CNAME: `dig solaris.health`
- Verify in registrar dashboard

“Frontend can’t reach backend”

- Check `VITE_API_URL` matches backend URL
 - Check CORS settings in `backend/src/server.js`
 - Test API directly: `curl https://solaris.health/api/health`
-

Support

- **Abacus.AI Docs:** abacus.ai/help (<https://abacus.ai/help>)
 - **SuperComputer Guide:** supercomputer.abacus.ai (<https://supercomputer.abacus.ai>)
 - **Community:** Discord/Slack (check Abacus.AI website)
-

Next Steps After Deployment

1. **Test with real users** (beta group)
 2. **Set up analytics** to track usage
 3. **Monitor performance** & optimize
 4. **Build roadmap features** (payments, FHIR export, scheduling)
 5. **Scale up** as user base grows
-

Ready to go live? Run `./deploy-abacus.sh` to containerize, then follow this guide to migrate to SuperComputer for always-on hosting. 🚀